

InternHub: Hệ Thống Phân Loại và Gợi Ý Phỏng Vấn IT

Cá Nhân Hóa Dựa Trên Knowledge Tracing

Mai Dương Hải Đăng
Trường ĐH Công nghệ Thông tin
ĐHQG-HCM
24520253@gm.uit.edu.vn

Tạ Gia Khiêm
Trường ĐH Công nghệ Thông tin
ĐHQG-HCM
24520805@gm.uit.edu.vn

Nguyễn Sỹ Huỳnh
Trường ĐH Công nghệ Thông tin
ĐHQG-HCM
24520716@gm.uit.edu.vn

Nguyễn Ngọc Nam
Trường ĐH Công nghệ Thông tin
ĐHQG-HCM
24521114@gm.uit.edu.vn

Abstract—Khóa luận trình bày *InternHub*, một hệ thống luyện phỏng vấn IT cá nhân hóa tích hợp Knowledge Tracing (KT) cho ba vai trò dữ liệu phổ biến: Data Analyst (DA), Data Scientist (DS) và Data Engineer (DE). Hệ thống giải quyết hai hạn chế của các nền tảng hiện nay: thiếu cá nhân hóa theo năng lực và không theo dõi quỹ đạo kiến thức của người học. Nhóm xây dựng question bank đa dạng (5 loại câu hỏi, 17 Knowledge Components) bằng pipeline lai giữa thu thập nguồn công khai và sinh tự động bằng mô hình ngôn ngữ lớn (LLM) kèm kiểm định chất lượng; đồng thời thiết kế pipeline mô phỏng dựa trên Item Response Theory (IRT) để tạo dữ liệu huấn luyện KT (1.000 người dùng ảo, ~40.000 tương tác). Ba họ mô hình KT—BKT, DKT, SAKT—được huấn luyện và so sánh, gồm biến thể *quality label* liên tục. Trên tập kiểm tra, mọi mô hình KT vượt baseline ($ROC - AUC \approx 0.68$ so với 0.50); DKT-Quality đạt $PR - AUC = 0.907$ và $RMSE = 0.355$, tốt nhất trong nhóm neural. Một cơ chế cập nhật năng lực thời gian thực kết hợp dự đoán KT (70%) với trung bình trượt mũ EMA (30%) được tích hợp vào engine gợi ý theo chiến lược *zone of proximal development* và bài kiểm tra thích nghi hai giai đoạn. Toàn bộ được hiện thực thành ứng dụng web hoàn chỉnh (React + FastAPI + Streamlit) với chấm điểm tự động bằng LLM và cơ chế chấm dự phòng cục bộ.

Index Terms—Knowledge Tracing, Deep Knowledge Tracing, Item Response Theory, hệ thống gợi ý, kiểm tra thích nghi, luyện phỏng vấn IT.

I. Giới Thiệu

Thị trường tuyển dụng nhân lực công nghệ thông tin tại Việt Nam và trên thế giới đang chứng kiến sự cạnh tranh ngày càng gay gắt. Theo khảo sát của TopDev **topdev2024**, hơn 70% ứng viên IT tại Việt Nam thừa nhận chưa chuẩn bị đủ kỹ năng kỹ thuật cần thiết trước các vòng phỏng vấn, trong khi phần lớn các nền tảng luyện phỏng vấn hiện có (LeetCode, HackerRank, Pramp) chủ yếu tập trung vào lập trình thuật toán, thiếu nội dung

dành riêng cho ba vai trò dữ liệu phổ biến: Data Analyst (DA), Data Scientist (DS) và Data Engineer (DE).

Hai thách thức cốt lõi tồn tại với các nền tảng hiện tại. Thứ nhất, **thiếu cá nhân hóa theo năng lực**: hầu hết hệ thống cung cấp câu hỏi theo cách tĩnh, không phản ánh được trình độ thực tế của từng ứng viên—người có kinh nghiệm phải làm lại các câu quá dễ, trong khi người mới bắt đầu nản lòng với câu hỏi ngoài tầm. Vấn đề này tương ứng với khái niệm *zone of proximal development* trong tâm lý học giáo dục **vygotsky1978mind**: câu hỏi hiệu quả nhất là câu hơi khó hơn năng lực hiện tại của người học một chút. Thứ hai, **không theo dõi được quỹ đạo kiến thức**: hệ thống hiện có không duy trì và cập nhật trạng thái kiến thức của người học theo thời gian, nên sau mỗi phiên luyện tập không biết người dùng đã tiến bộ ở điểm nào, còn yếu kỹ năng nào, và câu hỏi tiếp theo nên đi theo hướng nào.

Knowledge Tracing (KT) **corbett1994knowledge** là hướng giải quyết được nghiên cứu nhiều trong lĩnh vực Educational Data Mining **romero2010educational**, **baker2010data**, giúp mô hình hóa và dự đoán trạng thái kiến thức của người học qua lịch sử tương tác. Các mô hình KT từ Bayesian Knowledge Tracing **corbett1994knowledge** đến Deep Knowledge Tracing **piech2015deep** và Self-Attentive Knowledge Tracing **pandey2019self** đã chứng minh hiệu quả trong môi trường học thuật. Tuy nhiên, việc áp dụng KT cho bài toán luyện phỏng vấn IT—với đặc thù câu hỏi mở, chấm điểm tự động bằng LLM **team2023gemini** và nhiều loại câu hỏi (lý thuyết, coding, tình huống)—vẫn là một bài toán mở.

Khóa luận đặt ra bốn **mục tiêu**: (1) xây dựng bộ dữ liệu luyện KT phù hợp cho bài toán phỏng vấn IT; (2) huấn luyện và so sánh ba họ mô hình KT (BKT, DKT, SAKT) với biến thể nhân nhị phân và *quality label* liên tục; (3) xây dựng engine gợi ý dựa trên KT theo chiến lược

ZPD, có hỗ trợ kiểm tra thích nghi hai giai đoạn; (4) tích hợp vào ứng dụng web hoàn chỉnh với chấm điểm LLM và fallback cục bộ.

Phạm vi. Trong phạm vi: ba vai trò DA/DS/DE; 17 Knowledge Components (DA 5, DS 7, DE 5); năm loại câu hỏi (MC_SINGLE, TRUE_FALSE, FILL_BLANK, THEORY, CODING); dữ liệu KT mô phỏng gồm 1.000 người dùng, ~40.000 tương tác. Ngoài phạm vi: dữ liệu người dùng thật, triển khai production quy mô lớn, và A/B testing so với hệ thống thương mại.

Đóng góp chính. (1) Một pipeline xây dựng dữ liệu phỏng vấn IT tổng hợp, có thể tái sử dụng cho các bài toán KT nghề nghiệp khác; (2) biến thể DKT-Quality dùng nhân chất lượng liên tục, cải thiện RMSE và PR-AUC; (3) cơ chế blend KT(70%) + EMA(30%) với ngưỡng kích hoạt 3 tương tác, giải quyết cold-start; (4) hệ thống luyện phỏng vấn IT end-to-end tích hợp KT, kiểm tra thích nghi và chấm điểm LLM trong một codebase chạy local.

II. Công Trình Liên Quan

Knowledge Tracing. Từ BKT **corbett1994knowledge**, các mô hình KT đã phát triển qua DKT **piech2015deep**, DKVMN **zhang2017dynamic**, AKT **ghosh2020context**, và gần đây là các mô hình Transformer-based như SAINT **choi2020towards**. SAKT **pandey2019self** là mô hình attention đầu tiên đơn giản và hiệu quả, dựa trên cơ chế self-attention **vaswani2017attention**. Một số hướng mở rộng tích hợp nội dung bài tập (EKT **liu2019ekt**) hoặc cải thiện tính nhất quán của dự đoán DKT **yeung2018addressing**. Gần đây, một số nghiên cứu khảo sát khả năng của LLM trong giáo dục và đánh giá **liu2023llm**, **brown2020language**.

Hệ thống học thích nghi. Knewton **ferreira2017knewton** và ASSISTments **heffernan2014assistments** sử dụng KT để cá nhân hóa lộ trình học. Tuy nhiên, các hệ thống này tập trung vào môi trường học thuật, chưa có ứng dụng cụ thể cho phỏng vấn IT.

Hệ thống gợi ý. Các kỹ thuật như matrix factorization **koren2009matrix** và neural collaborative filtering **he2017neural** phổ biến trong recsys nói chung. *InternHub* khác biệt ở chỗ kết hợp KT với chiến lược ZPD thay vì chỉ dựa trên độ tương đồng giữa người dùng.

Hệ thống luyện phỏng vấn. Các nền tảng như LeetCode, HackerRank cung cấp bài tập lập trình nhưng thiếu cơ chế theo dõi kiến thức có cấu trúc. *InternHub* khác biệt ở chỗ tích hợp KT với question bank đa dạng (lý thuyết, thực hành, coding) cho ba vai trò DA/DS/DE, đồng thời sử dụng LLM để chấm điểm tự động.

III. Cơ Sở Lý Thuyết

A. Knowledge Tracing

Cho một chuỗi tương tác $\{(q_1, r_1), \dots, (q_t, r_t)\}$ trong đó q_t là câu hỏi tại thời điểm t và r_t là kết quả, mục tiêu

của KT là dự đoán xác suất trả lời đúng câu hỏi tiếp theo $p(r_{t+1} = 1 \mid q_{t+1}, q_{1:t}, r_{1:t})$. Trong *InternHub*, KT được áp dụng để theo dõi năng lực của ứng viên trên 17 Knowledge Components (KC) thuộc ba vai trò.

1) Bayesian Knowledge Tracing (BKT):

BKT **corbett1994knowledge** mô hình hóa từng KC như một chuỗi Markov ẩn (HMM) với hai trạng thái: *chưa thành thạo* và *thành thạo*, đặc trưng bởi bốn tham số cho mỗi KC: $P(L_0)$ (xác suất thành thạo ban đầu), $P(T)$ (chuyển trạng thái), $P(S)$ (slip—trả lời sai dù thành thạo), $P(G)$ (guess—trả lời đúng dù chưa thành thạo). Xác suất trả lời đúng tại bước t :

$$P(r_t = 1) = P(L_t)(1 - P(S)) + (1 - P(L_t))P(G). \quad (1)$$

Sau mỗi quan sát, trạng thái thành thạo được cập nhật theo Bayes:

$$P(L_t \mid r_t = 1) = \frac{P(L_t)(1 - P(S))}{P(r_t = 1)}, \quad (2)$$

$$P(L_{t+1}) = P(L_t \mid r_t) + (1 - P(L_t \mid r_t))P(T). \quad (3)$$

Tham số BKT được ước lượng bằng EM hoặc tối ưu trực tiếp L-BFGS-B trên log-likelihood. Ưu điểm: diễn giải cao, hiệu quả trên dữ liệu nhỏ; hạn chế: giả định KC độc lập.

2) Deep Knowledge Tracing (DKT):

DKT **piech2015deep** thay mô hình xác suất bằng mạng LSTM **hochreiter1997long**, cho phép học biểu diễn ẩn phong phú hơn:

$$\mathbf{h}_t = \text{LSTM}(\mathbf{x}_t, \mathbf{h}_{t-1}), \quad \hat{y}_{t+1} = \sigma(\mathbf{W}\mathbf{h}_t + \mathbf{b}). \quad (4)$$

InternHub mở rộng DKT với **quality label**: thay nhãn nhị phân $r_t \in \{0, 1\}$ bằng điểm chất lượng liên tục $r_t \in [0, 1]$ phản ánh mức độ thành thạo, chuyển hàm mất mát từ Binary Cross-Entropy sang MSE:

$$\mathcal{L}_{\text{quality}} = \frac{1}{T} \sum_{t=1}^T (r_t - \hat{y}_t)^2. \quad (5)$$

3) Self-Attentive Knowledge Tracing (SAKT):

SAKT **pandey2019self** thay kiến trúc hồi quy bằng self-attention **vaswani2017attention**, học mối liên hệ trực tiếp giữa các KC liên quan trong lịch sử. Attention score giữa câu q_t và bước lịch sử s :

$$a_{t,s} = \frac{(\mathbf{e}_{q_t} \mathbf{W}^Q)(\mathbf{m}_s \mathbf{W}^K)^\top}{\sqrt{d}}. \quad (6)$$

Ưu điểm: song song hóa (không tuần tự như LSTM) và diễn giải được attention weight.

B. Item Response Theory

IRT **embretson2000item**, **lord1980applications** mô hình hóa quan hệ giữa năng lực người học θ và xác suất trả lời đúng câu hỏi i . Mô hình 3PL dùng trong simulation:

$$P(r = 1 \mid \theta, a_i, b_i, c_i) = c_i + (1 - c_i) \frac{1}{1 + e^{-a_i(\theta - b_i)}}, \quad (7)$$

với a_i là độ phân biệt, b_i là độ khó, c_i là xác suất đoán. Trong *InternHub*, b ánh xạ từ EASY/MEDIUM/HARD thành $\{0.30, 0.55, 0.80\}$.

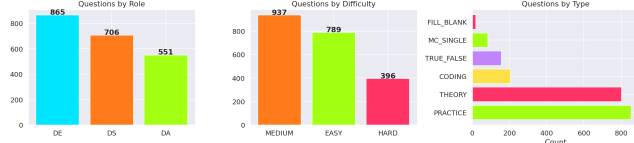


Figure 1: Phân bố câu hỏi theo vai trò và mức độ khó trong question bank.

C. Cập nhật năng lực thời gian thực: EMA và Blend

Mô hình KT cho dự đoán tốt khi đã đủ lịch sử, nhưng kém ổn định ở giai đoạn cold-start. *InternHub* kết hợp KT với Exponential Moving Average (EMA). Với mỗi KC, sau điểm $s_t \in [0, 1]$:

$$\theta_t^{\text{EMA}} = \alpha s_t + (1 - \alpha)\theta_{t-1}^{\text{EMA}}, \quad \alpha = 0.35. \quad (8)$$

EMA luôn chạy ngay từ tương tác đầu tiên. Khi lịch sử đạt tối thiểu 3 tương tác, dự đoán KT được blend với EMA:

$$\hat{\theta}_t = \lambda \theta_t^{\text{KT}} + (1 - \lambda)\theta_t^{\text{EMA}}, \quad \lambda = 0.70. \quad (9)$$

KT (trọng số trội) nắm bắt động lực chuỗi và quan hệ giữa các KC; EMA (30%) ổn định hóa dự đoán, chống dao động khi gặp chuỗi nhiễu. Hằng số λ và ngưỡng 3 tương tác được giữ nhất quán giữa backend và frontend để con số hiển thị đồng bộ trên toàn hệ thống.

IV. Xây Dựng Dữ Liệu

A. Thu thập và kiểm định Question Bank

Khác với phần lớn nghiên cứu KT chỉ dùng dataset có sẵn, khóa luận tự xây dựng question bank phỏng vấn IT qua một quy trình lại bốn giai đoạn. **Thu thập:** câu hỏi được scraping từ nhiều kho phỏng vấn mở (bộ DE của Obenner, bộ SQL của LearningZone, các bộ DS/DA cộng đồng), có trích dẫn nguồn, rồi ánh xạ sơ bộ về vai trò và nhóm kỹ năng. **Sinh tự động:** để tăng độ phủ các KC còn thiếu, một pipeline đa mô hình LLM được dùng—một mô hình sinh câu hỏi mới theo từng KC và mức độ, một mô hình mạnh hơn tạo đáp án tham chiếu và các *evaluation points*, một mô hình *judge* chấm chất lượng và một mô hình *enhancer* tinh chỉnh diễn đạt **team2023gemini**, **brown2020language**. **Kiểm định chất lượng:** mỗi câu đi qua kiểm tra schema, đối chiếu nội dung bằng RAG để phát hiện câu sai sự thật, và review thủ công trên mẫu ngẫu nhiên. **Chuẩn hóa:** ánh xạ các tag chi tiết (ví dụ SQL_WINDOW_FUNCTION) về 17 KC chuẩn qua bảng tra gồm 82 ánh xạ. Snapshot dùng cho thực nghiệm KT gồm 1.578 câu (17 KC, 3 mức độ); sau khi tiếp tục thu thập và sinh bổ sung, question bank của ứng dụng triển khai mở rộng lên 2.122 câu.

B. Mô phỏng tương tác bằng IRT

Do thiếu dữ liệu thực từ người dùng thật, nhóm sử dụng pipeline mô phỏng ba bước. **Bước 1:** sinh 1.000 người dùng ảo với skill vector theo phân phối correlated Gaussian, phản ánh tương quan thực tế giữa các competency (ví dụ DA có $r(\text{SQL}, \text{Analytics}) \approx 0.55$; DS

Table I: Thống kê bộ dữ liệu mô phỏng

Thuộc tính	Giá trị
Người dùng	1.000 (DA 334 / DS 333 / DE 333)
Câu hỏi (snapshot KT)	1.578
Knowledge Components	17
Tổng tương tác	40.140 (TB 40,1/user)
Train	700 users / 28.164 tương tác
Validation	150 users / 5.911 tương tác
Test	150 users / 6.065 tương tác
Pass rate ($Q \geq 1$)	82,5%

có $r(\text{Algorithm}, \text{Evaluation}) \approx 0.70$). **Bước 2:** với mỗi user, sinh chuỗi tương tác dựa trên IRT, $P(\text{correct}) = \sigma((\theta - b) \times 6)$. **Bước 3:** chấm theo bậc bằng power-based graded response, $P(Q=2) = p^\alpha$, $P(Q=0) = (1 - p)^\alpha$ với $\alpha = 2.0$, và cập nhật skill: $\theta += \eta(1 - \theta)$ khi đúng, $\theta -= \gamma\theta$ khi sai, với $\eta = 0.08$, $\gamma = 0.002$, seed 42.

C. Kiểm tra toàn vẹn dữ liệu

Trước huấn luyện, bộ dữ liệu được kiểm tra qua bảy tiêu chí: (1) không rò rỉ user giữa train và test; (2) phân phối nhân train (82,7%) $\approx$ test (82,2%), chênh lệch < 1%; (3) độ dài chuỗi đồng đều (trung bình 40 bước); (4) drift tần suất KC—2/17 KC có drift > 3%; (5) class imbalance 82,5% positive nên cần dùng PR-AUC thay vì chỉ ROC-AUC; (6) phân phối quality hợp lý; (7) 100% người dùng có cold-start vector.

V. Phân Tích Khám Phá Dữ Liệu

Phân tích người dùng. Phân bố người dùng đều theo vai trò (DA 334, DS 333, DE 333). Sau khi phân loại lại dựa trên skill vector thực, phân phối trình độ là beginner 40,9%, intermediate 53,2%, advanced 5,9%—lệch so với mục tiêu thiết kế (35/50/15%) do nhiều user advanced bị reclassify về intermediate khi mean skill nằm trong khoảng [0.40, 0.70]. Heatmap skill vector cho thấy tương quan dương rõ giữa các KC trong cùng vai trò.

Phân tích question bank. Question bank có phân bố khá đồng đều giữa ba vai trò, mỗi vai trò khoảng 500–550 câu; trong mỗi vai trò, câu MEDIUM chiếm tỷ lệ lớn nhất—phù hợp với thực tế phỏng vấn. Độ phủ theo KC có chênh lệch: các KC phổ biến như SQL_DATABASE (DA) và ALGORITHM_THEORY (DS) có nhiều câu hơn.

Phân tích tương tác. Phân phối quality score nhất quán với thiết kế IRT: $Q=2 \approx 62\%$, $Q=1 \approx 20\%$, $Q=0 \approx 18\%$ (Hình 2). Phân tầng theo độ khó xác nhận câu EASY có tỷ lệ $Q=2$ cao hơn rõ, câu HARD có $Q=0$ cao hơn; câu MEDIUM cân bằng nhất—vùng có độ phân biệt cao nhất trong IRT. Heatmap pass rate theo competency cho thấy các KC phức tạp như DEEP_LEARNING và NLP của DS có pass rate thấp nhất.

Sparsity và learning dynamics. Ma trận tương tác user-KC có cấu trúc block-diagonal do mỗi user chỉ trả lời câu thuộc vai trò của mình, dẫn đến sparsity $\approx 77\%$ trên toàn không gian KC nhưng chỉ $\approx 15\text{--}20\%$ trong từng vai trò—đủ để BKT hoạt động ổn định. Toàn bộ user có

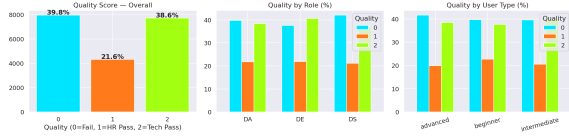


Figure 2: Phân phối quality score (0 = Fail, 1 = Pass HR, 2 = Pass Tech).

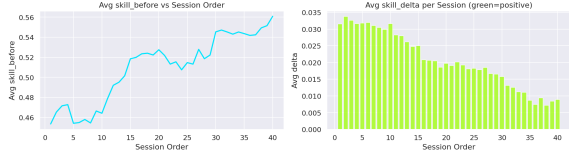


Figure 3: Learning curve: skill trung bình theo vị trí trong chuỗi tương tác.

đúng 40 tương tác do pool câu mỗi vai trò sau bộ lọc competency bị cap ở mức ≈ 40 câu. Learning curve tăng đơn điệu theo thứ tự câu (Hình 3), xác nhận động lực học đúng theo $\Delta\theta = \eta(1 - \theta)$: skill tăng nhanh hơn khi còn thấp. Phân tầng theo trình độ cho thấy nhóm beginner có tốc độ tăng skill tuyệt đối lớn nhất, nhóm advanced có hiệu ứng trần. Phân tích confidence bias cho hệ số tương quan $r(\text{self-rating}, \text{skill}) \approx 0.72$; nhóm beginner có xu hướng tự tin thái quá (overconfidence).

VI. Thực Nghiệm và Kết Quả

A. Môi trường và cấu hình

Thực nghiệm chạy trên Kaggle Notebooks (CPU Intel Xeon 2-core 13GB RAM, GPU NVIDIA Tesla T4 16GB, PyTorch 2.6 **pytorch2019**, Python 3.12, seed 42). Ablation study gồm tám cấu hình: Baseline (majority class), BKT (L-BFGS-B per KC), DKT-Binary và DKT-Quality* ($h=128$, $L=1$, dropout 0.2 **srivastava2014dropout**, lr 10^{-3}), SAKT-Binary và SAKT-Quality* ($e=64$, $H=4$, $L=1$), DKT-Deep ($h=256$, $L=2$) và SAKT-Deep ($e=128$, $L=2$). Mô hình neural dùng batch 64, tối đa 30 epoch, early stopping patience 5.

B. Độ đo

Nhóm dùng ROC-AUC (phân biệt đúng/sai, không phụ thuộc ngưỡng), PR-AUC (phù hợp khi mất cân bằng lớp), PR-AUC Gain ($= PR - AUC_{\text{model}} - PR - AUC_{\text{baseline}}$, đo cải thiện thực so với đường cơ sở), RMSE, và Accuracy. Do positive rate 82,5%, PR-AUC Gain là chỉ số so sánh chính.

C. Kết quả

Bảng II và Hình 4 cho thấy mọi mô hình KT vượt baseline rõ rệt: ROC-AUC tăng từ 0.50 lên 0.67–0.68, PR-AUC Gain đạt 0.083–0.087. Learning curves (Hình 5) cho thấy các mô hình neural hội tụ ổn định trong ~ 15 –19 epoch.

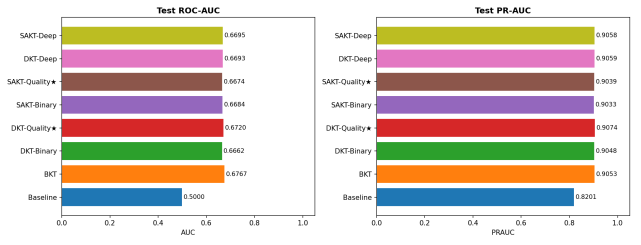


Figure 4: So sánh ROC-AUC và PR-AUC của 8 cấu hình.

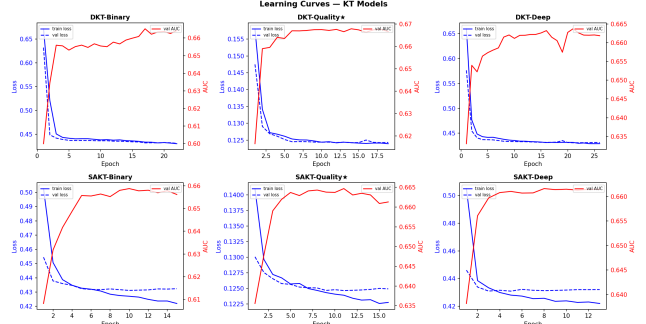


Figure 5: Learning curves của các mô hình neural (train/val loss, val ROC-AUC).

D. Phân tích

Hiệu quả KT. Tất cả mô hình KT vượt baseline, chứng minh lịch sử tương tác mang thông tin dự đoán thực sự ngay cả với dữ liệu mô phỏng.

Quality label. DKT-Quality* đạt PR-AUC cao nhất (0.907) và RMSE thấp nhất (0.355), vượt DKT-Binary (0.905, 0.375); nhân liên tục cung cấp tín hiệu gradient phong phú hơn. Accuracy thấp hơn (0.796 so với 0.822) do tối ưu MSE thay vì cross-entropy, làm phân phối đầu ra lệch khỏi ngưỡng 0.5.

Simulation bias. BKT đạt ROC-AUC cao nhất (0.677) vì dữ liệu được sinh theo IRT—về cơ bản cùng dạng tham số per-KC với BKT, tạo lợi thế không công bằng. Trên dữ liệu thật, mô hình neural được kỳ vọng tốt hơn nhờ học được các pattern phức tạp hơn.

Deep model. DKT-Deep (2 LSTM layer, 256 hidden) không cải thiện đáng kể so với DKT-Binary nhưng tốn thời gian gấp gần 4 lần (116s so với 31s); SAKT-Deep tương tự. Nút thắt là kích thước dữ liệu (28K tương tác train), không phải kiến trúc.

Hiệu quả tính toán. SAKT-Binary có thời gian huấn luyện ngắn nhất (11s) với ROC-AUC cạnh tranh, nhờ khả năng song song hóa của self-attention—phù hợp cho online update.

E. Phân tích sâu

Phân tích chi tiết bổ sung cho thấy: (i) theo KC (Hình 6), hiệu suất không đồng đều—KC tần suất thấp có ROC-AUC thấp hơn, và DKT-Quality* vượt BKT ở phần lớn KC thuộc DS/DE; (ii) theo vị trí, cả ba mô hình

Table II: Kết quả ablation study trên tập test (150 người dùng, 6.065 tương tác)

Mô hình	ROC-AUC	PR-AUC	PR-AUC Gain	RMSE	Acc	Time (s)
Baseline	0.500	0.820	0.000	0.384	0.820	0.0
BKT	0.677	0.905	0.085	0.374	0.820	61.9
DKT-Binary	0.666	0.905	0.085	0.375	0.822	30.8
DKT-Quality*	0.672	0.907	0.087	0.355	0.796	20.9
SAKT-Binary	0.668	0.903	0.083	0.373	0.822	11.3
SAKT-Quality*	0.667	0.904	0.084	0.356	0.776	12.0
DKT-Deep	0.669	0.906	0.086	0.375	0.822	116.4
SAKT-Deep	0.670	0.906	0.086	0.373	0.822	31.4

*: dùng quality label liên tục, tốt nhất trong nhóm neural.

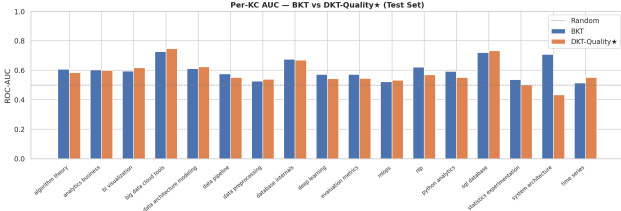


Figure 6: ROC-AUC theo từng KC: BKT vs DKT-Quality*.

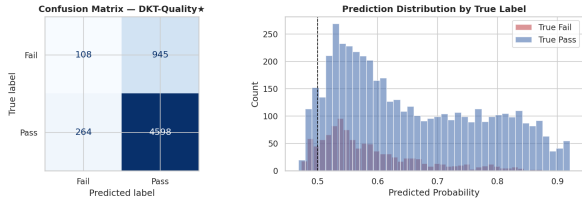


Figure 7: Confusion matrix và phân phối điểm dự đoán—DKT-Quality*.

cải thiện ROC-AUC khi có nhiều lịch sử hơn (early $\rightarrow$ late), xác nhận giá trị của việc thu thập nhiều tương tác; (iii) theo độ khó, câu MEDIUM có ROC-AUC cao nhất—vùng phân biệt cao nhất trong IRT; (iv) theo loại người dùng, nhóm intermediate dễ dự đoán nhất do phân phối đúng/sai cân bằng; (v) cold-start, cold-start score có tương quan dương yếu với độ chính xác per-user, xác nhận nó mang thông tin bổ sung nhưng không phải yếu tố duy nhất; (vi) phân tích lỗi (Hình 7), mô hình thiên về dự đoán Pass do class imbalance, và phân phối điểm dự đoán phân biệt được hai nhóm nhưng có chồng lấp do nhiễu mô phỏng.

F. Lựa chọn mô hình

Nhóm chọn DKT-Quality* làm mô hình chính cho ứng dụng: output dạng $(B, T, 17)$ cho phép dự đoán cả 17 KC trong một forward pass (quan trọng cho gợi ý thời gian thực), đồng thời đạt PR-AUC/RMSE tốt nhất nhóm neural và train nhanh ($\approx 21s$). SAKT-Binary là lựa chọn cho online update nhanh khi cần.

VII. Thiết Kế và Hiện Thực Hệ Thống

A. Kiến trúc ba tầng

InternHub được tổ chức thành ba tầng. Tầng dữ liệu chứa question bank, JSON schema và taxonomy 17 KC. Tầng KT/scoring gồm kt_predictor (nạp DKT-Quality), recommender và competency_engine. Tầng phục vụ gồm FastAPI fastapi2018 (9 endpoint /api/*), Streamlit streamlit2019 làm host và React SPA react2013. Backend (snake_case) và frontend (camelCase) dùng quy ước đặt tên và lược đồ câu hỏi khác nhau; mọi phép chuẩn hóa được tập trung tại ranh giới ở hai điểm: _map_question() (Python $\rightarrow$ React) và _normalize_question() (React $\rightarrow$ Python).

B. Engine năng lực và gợi ý

competency_engine định nghĩa taxonomy 17 KC theo vai trò và bảng tra SKILL_GROUP_TO_KC (82 ánh xạ), cùng hai ngưỡng phân loại: KC yếu nếu $\theta < 0.40$, KC mạnh nếu $\theta \geq 0.70$. Engine gợi ý xếp hạng câu hỏi theo điểm tổng hợp. Khi tập trung KC yếu:

$$\text{score}(q) = 0.65 \cdot \text{weakness} + 0.30 \cdot \text{diff_fit} + 0.05 \cdot \text{coverage}, \quad (10)$$

trong đó $\text{weakness} = \max(0, 0.40 - \theta_{kc})/0.40$, diff_fit đo độ phù hợp độ khó với năng lực (chiến lược ZPD), coverage thưởng cho phủ KC chưa mạnh. Ở chế độ thử thách, trọng số đổi thành $0.45 \text{ readiness} + 0.45 \text{ diff_fit} + 0.10 \text{ coverage}$. Khi chạy, kt_predictor nạp DKT-Quality (singleton) và dự đoán mastery $\in [0.05, 0.99]$; nếu lịch sử < 3 hoặc không nạp được model thì lui về EMA thuần (graceful fallback).

C. Kiểm tra thích nghi hai giai đoạn

Phiên Diagnostic áp dụng CAT hai giai đoạn wainer2000computerized. Stage 1 chọn câu phủ đều các KC của vai trò; điểm trung bình quyết định routing WEAK/MID/STRONG. Stage 2 chọn câu chuyên sâu, ưu tiên các KC yếu nhất phát hiện ở Stage 1, độ khó điều chỉnh theo nhánh (STRONG ưu tiên ADVANCED; WEAK ưu tiên INTERMEDIATE để học). Vector năng lực cập nhật theo từng trục KC—chỉ trục được kiểm tra mới đổi; lần đánh giá đầu tiên ánh xạ trực tiếp điểm số thành giá trị KC. Điểm cuối dùng pipeline tiếp điểm số thành giá trị KC. Điểm cuối dùng pipeline KT-EMA chung (Công thức (9)) nên radar ở màn kết quả, Dashboard và màn KT Model đều hiển thị cùng một con số.

D. Hệ thống chấm điểm

Câu khách quan (MC_SINGLE, TRUE_FALSE, FILL_BLANK) được chấm tất định phía client, không gọi LLM. Câu tự luận và coding (THEORY, PRACTICE, CODING) được chấm bằng Gemini **team2023gemini** với prompt chứa câu hỏi, đáp án tham chiếu và evaluation points. Khi Gemini lỗi (rate limit, mạng), hệ thống lui về local rubric: một evaluation point được coi là “đạt” khi $\geq 50\%$ từ khóa của nó xuất hiện trong câu trả lời; nếu không có eval points thì dùng độ tương đồng Jaccard từ khóa so với đáp án tham chiếu.

E. Tầng phục vụ và giao diện

Streamlit không phục vụ tốt tệp tĩnh với đúng MIME type, nên UI thực không do Streamlit phục vụ: `app.py` khởi động FastAPI ở thread nền (cổng 8000), FastAPI mount thư mục `static/` qua `StaticFiles(html=True)`; trang Streamlit chỉ là một `<iframe>`; React gọi ngược về `/api` qua `fetch`. Frontend gồm các tệp JSX nạp tuần tự bằng thẻ `<script type="text/babel">` và biên dịch ngay trên trình duyệt bằng Babel standalone—không cần npm/webpack; mọi tệp chia sẻ một global scope và dữ liệu question bank được serialize từ Python qua `window.__INTERHUB_DATA__`.

VIII. Hạn Chế và Thảo Luận

Simulation bias (nghiêm trọng nhất). Toàn bộ dữ liệu huấn luyện KT được sinh bởi IRT, tạo lợi thế không công bằng cho BKT do cùng inductive bias; kết quả trên dữ liệu người dùng thực có thể khác biệt đáng kể. Đây là hạn chế căn bản nhất cần giải quyết để đưa hệ thống vào production.

Sequence length cố định. Pool câu mỗi vai trò sau bộ lọc competency chỉ ≈ 40 câu, khiến mọi user có đúng 40 tương tác; mô hình không được huấn luyện trên sequence dài ngắn khác nhau, làm giảm khả năng tổng quát hóa sang người dùng thật.

Class imbalance. Pass rate 82,5% làm PR-AUC bị inflate; PR-AUC Gain (≈ 0.085) là chỉ số đáng tin hơn để so sánh.

Phân phối advanced thấp. Chỉ 5,9% người dùng advanced sau reclassification, nên mô hình ít được expose với người dùng skill cao.

Phụ thuộc LLM. Khi Gemini không khả dụng, fallback local rubric cho điểm kém chính xác hơn; chưa có cơ chế cache/queue để retry, và chưa calibrate điểm Gemini với human annotation.

Scalability. FastAPI chạy trong background thread của Streamlit là giải pháp tạm cho demo local; production cần tách microservice, thêm DB và authentication.

IX. Kết Luận và Hướng Phát Triển

Khóa luận đã thiết kế và hiện thực *InternHub*—hệ thống luyện phỏng vấn IT cá nhân hóa dựa trên Knowledge Tracing cho ba vai trò DA/DS/DE. Từ việc xây dựng bộ dữ liệu tổng hợp (thu thập + LLM + QC + mô phỏng

IRT), huấn luyện và so sánh tám cấu hình mô hình KT, đến tích hợp end-to-end vào ứng dụng web với chấm điểm LLM, mỗi bước đều được trình bày kèm phân tích. Kết quả xác nhận KT mang lại giá trị thực ($ROC - AUC \approx 0.68$ so với 0.50); DKT-Quality với quality label liên tục cho kết quả tốt nhất nhóm neural; và cơ chế blend KT-EMA giải quyết được bài toán cold-start mà các hệ thống KT truyền thống thường bỏ qua.

Hạn chế lớn nhất là thiếu dữ liệu thực từ người dùng—vừa là giới hạn hiện tại, vừa là động lực cho hướng phát triển. Các hướng tiếp theo gồm: (1) thu thập dữ liệu thực và A/B test để loại bỏ simulation bias; (2) tích hợp forgetting curve **ebbinghaus1885memory** và spaced repetition **wozniak1990optimization**; (3) thử các mô hình Transformer đầy đủ như AKT **ghosh2020context** và SAINT **choi2020towards** khi đủ dữ liệu; (4) gợi ý đa mục tiêu **van1998optimal**; (5) mở rộng question bank và triển khai production với data flywheel.